

TicAmazon: decisiones de diseño del servidor de productos

Hilos o procesos

El servidor atiende cada solicitud con un hilo nuevo (`std::thread`), no con un proceso nuevo. Se eligió este modelo porque las solicitudes de distintos clientes necesitan leer y escribir sobre el mismo archivo de almacenamiento, `cafeteria.dat`, y con hilos ese archivo se puede compartir directamente a través de una sola instancia de la clase que lo maneja. Con procesos, cada uno tendría su propio espacio de memoria y su propio descriptor de archivo abierto por separado, lo que hubiera obligado a coordinar el acceso al disco con mecanismos entre procesos en vez de un simple mutex en memoria.

La sección crítica de cada operación de almacenamiento (pedir un bloque, leer o escribir un bloque, seguir el puntero de un directorio) queda protegida con un mutex dentro de la clase `FileSystem`. Como todos los hilos del proceso comparten ese mutex, dos solicitudes que lleguen al mismo tiempo nunca hacen la misma escritura; una espera a que la otra termine su operación antes de continuar.

Servidor integrado de productos e intermediario

El protocolo de la asignación define al Intermediario y al servidor de productos (SERV) como dos entidades de red separadas, cada una con su propia dirección, que se comunican con mensajes de texto sin usar HTTP. Para esta etapa se solicitó, implementarlos como un solo proceso: un único binario (intermediario) que escucha las conexiones HTTP de los clientes y, dentro de esa misma llamada, resuelve la solicitud contra el almacenamiento de las bodegas.

El protocolo entre Intermediario y Bodega no está sujeto al requisito de usar HTTP precisamente porque se asume que puede ser una comunicación directa.

Para no perder la trazabilidad del protocolo, cada solicitud que en el diseño original hubiera viajado como un mensaje hacia SERV (por ejemplo `REQUEST_CATEGORY` o `REQUEST_PRODUCTS`) se sigue registrando en la salida de consola del Intermediario, aunque internamente sea una llamada directa a un método de la clase Bodega en vez de un mensaje que cruza un socket.

Definición de las bodegas

Se definieron dos bodegas, cada una responsable de un subconjunto fijo de categorías del negocio: Bodega-1 maneja Bebidas frías, Bebidas calientes y Postres; Bodega-2 maneja Caramelos y Repostería. Esta asignación de categorías

a bodega no vive en el disco, vive en el código que arranca el sistema (en el main del Intermediario), siguiendo el mismo principio que ya se había definido para el modelo de almacenamiento: el archivo no necesita saber qué bodega es dueña de qué categoría.

Cada bodega está representada por una instancia de la clase Bodega, que no tiene su propio archivo en disco: las dos comparten un único archivo, cafeteria.dat, de 1024 bloques de 256 bytes. Lo que distingue a una bodega de otra dentro de ese archivo es el bloque donde arranca su propio directorio de categorías, un número que la clase FileSystem le entrega a cada Bodega en el momento de registrarla (FileSystem::registrarBodega). A partir de ahí, cada Bodega solo lee y escribe los bloques de sus propias categorías, índices y productos; nunca toca los bloques de la otra bodega, aunque ambas puedan pedir bloques nuevos del mismo mapa de bits.

Llenado de datos de productos

Los datos de ejemplo se cargan de forma automática cada vez que arranca el Intermediario, como parte de la inicialización del sistema, antes de que el proceso empiece a escuchar conexiones. El main crea el archivo cafeteria.dat, registra las dos bodegas, crea las categorías de cada una con Bodega::crearCategoria, y agrega los productos iniciales con Bodega::agregarProducto, indicando nombre, cantidad disponible y precio en centavos.

Se optó por cargar estos datos directamente en código, en vez de leerlos de un archivo de configuración externo, porque el catálogo de esta entrega es pequeño y fijo, y mantenerlo en el propio main hace explícito, con una sola lectura del archivo, qué productos existen y en qué bodega están. Como cafeteria.dat se recrea desde cero en cada arranque del Intermediario (FileSystem::crearArchivo trunca el archivo), esta carga de datos es también lo que garantiza que cada corrida de pruebas empiece siempre con el mismo catálogo conocido, sin productos ni categorías que hayan quedado de una corrida anterior.